

The Everything Data Model

Abstract

In this paper we outline the model humans use to describe the world and formalize it. This will enable us to store raw knowledge in an unambiguous, defined format.

Things

Humans think in things. Everything we can think of is a thing. There is a set of things. The things themselves are just abstract; what is important is the semantics, the meaning of them. In everyday conversations, we as humans talk about things. And you can only talk about a thing if you can identify it. And if you cannot talk about a thing because you cannot identify it then it is not a thing because we could not think of it. Therefore there can only exist so many things as identifiers. In this paper we will use *object* as a synonym for thing.

Identifying Things

The straightforward way for us humans to talk about a thing are names. For example, we identify people by their names, places, activities, etc. The thing is (pun intended) that names can be ambiguous.

A person *R* growing up with banks as in river banks may ought to uniquely and unambiguously identify the concept of river banks with the word “bank”. A sales person *S* in the city may use the word “bank” to uniquely and unambiguously identify the bank as in where you can store money. A conversation of banks between may lead to a common but interesting phenomenon. *R* might say “Banks are interesting!” and *S* would agree, as *S* could understand how one might find banks (as in where you deposit money) interesting.

The interesting point is where *S* would say: “Yeah, they store so much money”. *R*: “Money? At the river?”. And this is the tipping point. *R* realizes that the property of banks described by *S* is inapplicable to the object referenced by *R*. Both parties realize they have apparently been talking about different objects. They now have to adjust their object name / identifier because they have been made aware of the existence of a another similarly named object.

Now, *S*: “Yeah, you know? The banks that store money.” *R*: “Oh there banks are that store money? I thought banks were the sides of a river..” *S*: “Well, these are called banks.” Now the names have changed to “river bank” and “money bank”.

Semantic Meaning

Identifiers – like words – are nice because they partly describe what the thing is. This makes it easy for us humans to remember them. However, this may lead to problems if the meaning ever changes.

Consider the example of two Michael Jacksons. To distinguish them, one may be called “the Michael Jackson that is at Remix” and the other one “Michael Jackson, the black one”. But Michael Jackson, the musician, transitioned into white. The person stayed the same but he is not black anymore.

Meanings change, so names are not a good fit for identifiers. Only if we decouple the semantics from the id, there is nothing to misinterpret. Numbers, UUIDs, or ULIDs are a good fit.

Data

When we as humans talk about numbers or other forms of data, we usually do not reference them via a name but just write down the number we mean. There is *basically* no ambiguity when talking about numbers but more so the interpretation of them. 42 and 42 refer to the same object, the number 42. This is a different kind of object reference that is identified by itself.

Relating Things

...

Mathematical Description

Upfront there is a distinction to make between the theoretical model – the *Everything data model* (EDM) – and the implementation – the *Everything database*.

The EDM works with objects, states facts, and places them in relation to each other. To address and talk about the objects, the EDM uses a domain of object references / identifiers (ids). There are two distinct types of object ids: *abstract object ids* (AOIs) with the set O_A and *data object ids* (DOIs) defined by the set O_D . These sets must be mutually exclusive: $O_A \cap O_D = \emptyset$. The set of all object ids is: $O := O_A \cup O_D$.

Abstract Object Ids

An abstract object id is an opaque object reference that unambiguously identifies the object referenced whose meaning in the model is in no correlation with the id. Abstract object ids therefore do not carry any semantic information about the object and exist solely to identify objects.

One may choose $O_A := \{ @n \mid n \in \mathbb{N}_0 \}$ as their domain. The Everything database implementation uses 127-bit ULIDs and therefore the maximum number of abstract objects which can be referenced is 2^{127} (which is reasonable). This design decision is due to technical limitations.

Data Object Ids

A data object id unambiguously references the object that is the data itself. The “data” is just an integer. Therefore, O_D is the set of all data values the model can handle.

One may choose $O_D := \mathbb{N}_0$.

Associations

The EDM’s primitive is not the object id but the connections between the objects. An *association* is a three-tuple of object ids. Conceptually, associations connect objects with the following semantics:

- The first tuple entry is nicknamed *target*. It represents the object in question, the object stated about in the association.
- The second tuple entry is nicknamed *tag*. It represents the property of the target stated about in this association.
- The third (and final) tuple entry is nicknamed *value*. It represents the value the property has on the target.

The set of all associations is defined as follows: $A := O \times O \times O$.

Builtin Objects

The EDM is a self-restricting model. This will become apparent when defining the database in the following section. But for that it needs some builtin anchors such that it can interface with the implementation of the database which does the actual validation.

Therefore, we define the following set of builtin objects which are referenced by these ids. The M stands for “meta” but does not have any role beyond that.

- $M_{\text{tag}} \in O_A$
- $M_{\text{inferred}} \in O_A$
- $M_{\text{type.0}} \in O_A$

All these symbols reference different objects. Also, let $0 \in O_D$ be the data object that represents no data.

Databases

A database (also called knowledge base) is a finite set of associations ($D \subseteq A \wedge |D| \neq \infty$) that satisfies constraints (1) to (9).

An object a is *tagged* with an object b and a value c if there exists an association $(a, b, c) \in D$.

- (1) Every tag used in an association with a value v must be tagged with M_{tag} and a value T such that T is tagged with M_{tag} and value $M_{\text{type}.0}$, and v must be tagged with T and a value of 0. Here is the formal constraint:

$$(o, t, v) \in D \implies \exists T \in O \wedge (t, M_{\text{tag}}, T) \in D \wedge (T, M_{\text{tag}}, M_{\text{type}.0}) \in D \wedge (v, T, 0) \in D.$$

The motivation behind this constraint is that you cannot arbitrarily tag objects with other objects. You have to declare the tag object as a tag. By doing this, you also constraint the objects that can be used as values in the association by defining a “type”. This type is represented by the variable T in the constraint, and its purpose is only to tag objects as values for this type. Tagging an object to be of type T does not require any value, hence the $M_{\text{type}.0}$ in $(T, M_{\text{tag}}, M_{\text{type}.0})$. And finally, the value actually used in the association must adhere to the type T ; hence $(v, T, 0) \in D$.

...

The empty set $\emptyset$ is a database (?).

Core Associations

Interesting behavior emerges from these constraints. For this we assume a non-empty database $D \neq \emptyset$.

$$\begin{aligned} D &\neq \emptyset \\ \implies \exists (o, t, v) &\in D \\ \stackrel{(1)}{\implies} \exists T \in O &\wedge (t, M_{\text{tag}}, T) \in D \wedge (T, M_{\text{tag}}, M_{\text{type}.0}) \in D \wedge (v, T, 0) \in D \end{aligned}$$

...

Working only with M_{tag} :

$$\begin{aligned} \exists T \in O &\wedge (t, M_{\text{tag}}, T) \in D \\ \stackrel{(1)}{\implies} \exists T' \in O &\wedge (M_{\text{tag}}, M_{\text{tag}}, T') \in D \wedge (T', M_{\text{tag}}, M_{\text{type}.0}) \in D \wedge (T, T', 0) \in D \end{aligned}$$

...

Solution

$$\begin{aligned} D := \{ &(M_{\text{tag}}, M_{\text{tag}}, M_{\text{type}}), (M_{\text{type}}, M_{\text{tag}}, M_{\text{type}.0}), (M_{\text{type}}, M_{\text{type}}, 0), \\ &(M_{\text{type}.0}, M_{\text{type}}, 0), (M_{\text{type}.0}, M_{\text{tag}}, M_{\text{type}.0}), (0, M_{\text{type}.0}, 0) \} \end{aligned}$$

is a database.